Validación de requerimientos

Cuando hablamos de validación de requerimientos, debemos tener en claro algunos conceptos que constituyen el desarrollo del *software*.
Debemos identificar los diferentes elementos que componen la validación de requerimientos y tener en cuenta el desarrollo de diferentes tipos de verificación y pruebas que tenemos disponibles.

- Caso de análisis
- Validación de requerimientos
- Testing
- Video conceptual
- Referencias

Caso de análisis

Una cadena de cabañas permite realizar la reserva de las habitaciones vía web. La persona interesada reservará un tipo de habitación por un periodo determinado, que posee una tarifa que varía según la temporada (alta, media o baja). En el momento de la reserva, no deberá abonar nada, pero sí deberá hacerlo cuando la confirme, un mes antes del periodo elegido. La cadena tiene un catálogo con los tipos de cabañas con que cuenta y las tarifas de acuerdo con la temporada, que enviará dos veces por año a los clientes para que puedan consultarlo. En el momento de llegada al complejo, se asigna el número de cabaña.

La cadena de cabañas se encuentra en toda la Argentina y necesita, en cualquiera de las sucursales, poder realizar reservas y generar el *check in* y el *check out* de cada cliente.

Las cabañas tienen, actualmente, una tecnología que quedó desactualizada con un procesamiento inadecuado de los volúmenes de información con un lenguaje de programación inadecuado, ya que existen diferentes lenguajes de programación para procesar la información que influyen en el funcionamiento de la página.

El equipo de sistemas de la cadena de cabañas decidió desarrollar los siguientes sistemas:

- Necesita que el desarrollo de la página web sea rápido y pueda ver avances en tiempos cortos utilizando un ciclo de vida a elegir.
- Necesita que la página web se pueda ir actualizando con diferentes versiones a medida que la tecnología evolucione y que se utilice PHP para el diseño de la página web.
- Necesita que el software pueda recibir la visita de 1000 clientes en dos horas.
- Necesita tener un disco rígido para almacenar la información de por lo menos 1 *terabyte*.
- Necesita un sistema de reservas que permita identificar un análisis de riesgos detallado, que pueda identificar los elementos que contiene el sistema de reservas y pueda realizar un relevamiento dentro de la empresa a los responsables del área de reservas la información obtenida en el relevamiento, complementando la información con lo obtenido de 200 empleados que se encuentran relacionados con el área de reservas.
- Necesita registrar los requerimientos en un documento gráfico detallado y una herramienta gráfica.
- Necesita que se pueda identificar cada interfaz en forma sencilla y que no falte ningún elemento.
- Necesita poder registrar, modificar, consultar, anular e imprimir las reservas.
- Necesita desarrollar un sistema de facturación donde necesita tener la información de 25 personas que permita facturar con un IVA del 25 % para clientes que vienen de Brasil.
- Necesita poder contemplar los riesgos que comprenden el sistema de facturación para saber las medidas a aplicar.

- Necesita desarrollar un sistema de movimientos diarios de dinero, que utilice para la construcción del ciclo de vida secuencial por la claridad de los elementos que componen el sistema.

Actualmente, se viene desarrollando un sistema de cobranzas donde se están probando los diferentes componentes del sistema con su código correspondiente.

El sistema de *check in* se encuentra terminando la versión 1.1 para poder probarla en el sistema operativo requerido, ya que necesita que se utilice como base de datos la SQL y se puedan compartir recursos en la red que tiene el hotel.

Una vez integrado el sistema de *check in* debe contemplar que funcione bien ante 100 solicitudes de *check in* de 8 a 10 h y que cada registro, consulta e impresión funcione adecuadamente.

En el sistema de *check in,* se encuentra realizando *testing* de fallas de datos, fallas de control, fallas de entrada/salida, fallas de interfaz y fallas de gestión de almacenamiento.

Aún se necesita que el sistema de reservas sea aprobado por los usuarios para verificar que funcione sin errores.

También, se necesita que la empresa tenga una impresora láser color y que funcione bajo el sistema operativo Windows.

La cadena de cabañas se encuentra distribuida en las regiones del norte, sur este y oeste y cuenta con 5 hoteles en cada una de ellas.

Siguiendo con la problemática planteada, vamos a desarrollar los conceptos de validación de requerimientos del sistema de *check in*

CONTINUAR

Validación de requerimientos

La validación de requerimientos es el proceso de verificar que los requerimientos definan realmente el sistema que en verdad quiere el cliente. Se traslapa con el análisis, ya que se interesa por encontrar problemas con los requerimientos. La validación de requerimientos es importante porque los errores en un documento de requerimientos pueden conducir

a grandes costos por tener que rehacerlo, cuando dichos problemas se descubren durante el desarrollo del sistema o después de que este se halla en servicio.

En general, el costo por corregir un problema de requerimientos al hacer un cambio en el sistema es mucho mayor que reparar los errores de diseño o codificación. La razón es que un cambio a los requerimientos significa, generalmente, que también deben cambiar el diseño y la implementación del sistema. Más aún, el sistema debe entonces ponerse a prueba de nuevo.

Durante el proceso de validación de requerimientos, tienen que realizarse diferentes tipos de comprobaciones sobre los requerimientos contenidos en el documento de requerimientos.

Dichas comprobaciones incluyen las siguientes:

1. **Comprobaciones de validez.** Un usuario quizá crea que necesita un sistema para realizar ciertas funciones. Sin embargo, con mayor consideración y análisis se logra identificar las funciones adicionales o diferentes que se requieran. Los sistemas tienen diversos participantes con diferentes necesidades, y cualquier conjunto de requerimientos es inevitablemente un compromiso a través de la comunidad de participantes

2. **Comprobaciones de consistencia**. Los requerimientos en el documento no deben estar en conflicto. Esto es, no debe haber restricciones contradictorias o descripciones diferentes de la misma función del sistema.

3. **Comprobaciones de totalidad.** El documento de requerimientos debe incluir requerimientos que definan todas las funciones y las restricciones pretendidas por el usuario del sistema.

4. **Comprobaciones de realismo**. Al usar el conocimiento de la tecnología existente, los requerimientos deben comprobarse para garantizar que en realidad pueden implementarse.

Dichas comprobaciones también tienen que considerar el presupuesto y la fecha para el desarrollo del sistema

5 **Verificabilidad.** Para reducir el potencial de disputas entre cliente y contratista, los requerimientos del sistema deben escribirse siempre de manera que sean verificables.

Esto significa que se debe ser capaz de escribir un conjunto de pruebas que demuestren que el sistema entregado cumpla cada requerimiento especificado.

Hay algunas **técnicas de validación** de requerimientos que se usan individualmente o en conjunto con otras:

1. Revisiones de requerimientos. Los requerimientos se analizan sistemáticamente usando un equipo de revisores que verifican errores e inconsistencias.

2. Creación de prototipos. En esta aproximación a la validación, se muestra un modelo ejecutable del sistema en cuestión a los usuarios finales y clientes. Así, ellos podrán experimentar con este modelo para constatar si cubre sus necesidades reales.

3. Generación de casos de prueba. Los requerimientos deben ser comprobables. Si las pruebas para los requerimientos se diseñan como parte del proceso de validación, esto revela con frecuencia problemas en los requerimientos. Si una prueba es difícil o imposible de diseñar, esto, generalmente, significa que los requerimientos serán difíciles de implementar, por lo que deberían reconsiderarse. El desarrollo de pruebas a partir de los requerimientos del usuario antes de escribir cualquier código es una pieza integral de la programación extrema.

No hay que subestimar los problemas incluidos en la validación de requerimientos.

A fin de cuentas, es difícil demostrar que un conjunto de requerimientos no cubre las necesidades de los usuarios. Estos últimos necesitan una imagen del sistema en operación, así como comprender la forma en que dicho sistema se ajustará a su trabajo.

Es difícil, incluso para profesionales de la computación experimentados, realizar este tipo de análisis abstracto y, más aún, para los usuarios del sistema. Como resultado, rara vez encontrarás todos los problemas de requerimientos durante el proceso de validación de requerimientos. Es inevitable que haya más cambios en los requerimientos para corregir omisiones y malas interpretaciones, después de acordar el documento de requerimientos.

Siguiendo con la problemática planteada, vamos a desarrollar los conceptos de validación de cada componente del sistema de *check in*.

Validando las pruebas

Las pruebas no pueden demostrar que el software está libre de defectos o que se comportará en todo momento como está especificado. Siempre es posible que una prueba que se haya pasado por alto pueda descubrir problemas adicionales con el sistema.

Generalmente, por lo tanto, el objetivo de las pruebas del *software* es convencer a los desarrolladores del sistema y a los clientes de que el software es lo suficientemente bueno para su uso operacional. La prueba es un proceso que intenta proporcionar confianza en el *software*.

Los casos de prueba son especificaciones de las entradas para la prueba y la salida esperada del sistema más una afirmación de lo que se está probando. Los datos de prueba son las entradas que han sido ideadas para probar el sistema. Los datos de prueba a veces pueden generarse automáticamente.

La generación automática de casos de prueba es imposible. Las salidas de las pruebas solo pueden predecirse por personas que comprenden lo que debería hacer el sistema.

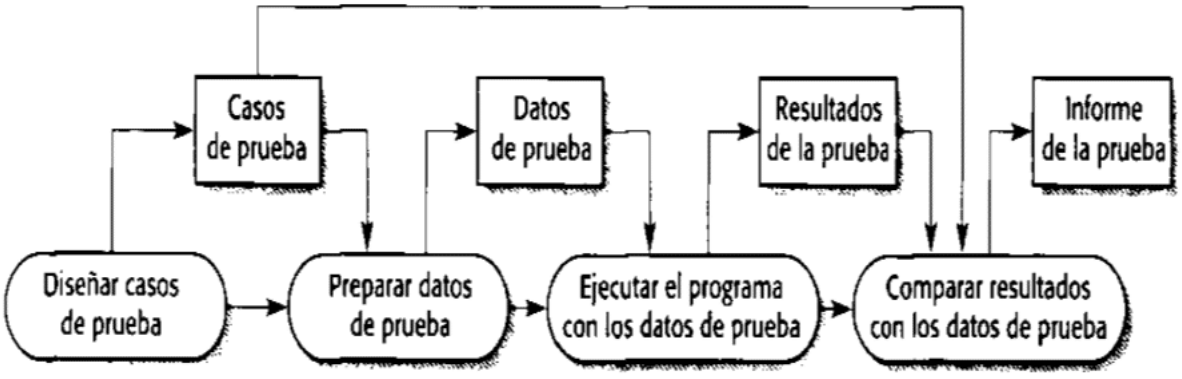
Las pruebas exhaustivas, en las que cada posible secuencia de ejecución del programa es probada, son imposibles. Las pruebas, por lo tanto, tienen que basarse en un subconjunto de posibles casos de prueba. Idealmente, algunas compañías deberían tener políticas para elegir este subconjunto en lugar de dejar esto al equipo de desarrollo. Estas políticas podrían basarse en políticas generales de pruebas, tal como una política en la que todas las sentencias de los programas deberían ejecutarse al menos una vez. De forma alternativa, las políticas de pruebas pueden basarse en la experiencia de uso del sistema y pueden centrarse en probar las características del sistema operacional. Por ejemplo:

1. Deberían probarse todas las funciones del sistema a las que se accede a través de menús.

2. Deben probarse todas las combinaciones de funciones (por ejemplo, formateado de textos) a las que se accede a través del mismo menú.

3. En los puntos del programa en los que el usuario introduce datos, todas las funciones deben probarse con datos correctos e incorrectos

Figura 1: Modelo del proceso de pruebas del *software*



Fuente: Sommerville, 2005, p. 493

Pruebas del sistema

Para la mayoría de los sistemas complejos, existen dos fases distintas de pruebas del sistema:

1. Pruebas de integración, en las que el equipo de pruebas tiene acceso al código fuente del sistema. Cuando se descubre un problema, el equipo de integración intenta encontrar la fuente del problema e identificar los componentes que tienen que ser depurados.

Las pruebas de integración se ocupan, principalmente, de encontrar defectos en el sistema.

2. Pruebas de entregas, en las que se prueba una versión del sistema que podría ser entregada a los usuarios. Aquí, el equipo de pruebas se ocupa de validar que el sistema satisfaga sus requerimientos y asegurar que el sistema sea confiable.

Las pruebas de entregas son, normalmente, pruebas de «caja negra» en las que el equipo de pruebas se ocupa simplemente de demostrar si el sistema funciona o no correctamente. Los problemas son comunicados al equipo de desarrollo, cuyo trabajo es depurar el programa.

Cuando los clientes se implican en las pruebas de entregas, estas, a menudo, se denominan **pruebas de aceptación**. Si la entrega es lo suficientemente buena, el cliente puede entonces aceptarla para su uso.

Fundamentalmente, se puede pensar en las pruebas de integración como las pruebas de sistemas incompletos compuestos por grupos de componentes del sistema.

Las pruebas de entregas consisten en probar la entrega del sistema que se pretende proporcionar a los clientes.

Naturalmente, estas se solapan, en especial, cuando se utiliza un desarrollo incremental y el sistema está incompleto para entregar. Generalmente, la prioridad en las pruebas de integración es descubrir defectos en el sistema, y la prioridad en las pruebas del sistema es validar que el sistema satisfaga sus requerimientos. Sin embargo, en la práctica, hay una parte de prueba de validación y una parte de prueba de defectos durante ambos procesos

Pruebas de integración

La integración del sistema implica identificar grupos de componentes que proporcionan alguna funcionalidad del sistema e integrar estos añadiendo código para hacer que funcionen conjuntamente.

Algunas veces, primero se desarrolla el esqueleto del sistema en su totalidad, y se le añaden los componentes. Esto se denomina i**ntegración descendente.**

De forma alternativa, pueden integrarse primero los componentes de infraestructura que proporcionan servicios comunes, tales como **el acceso a bases de datos y redes** y, a continuación, pueden añadirse los componentes funcionales.

Esta es la **integración ascendente**.

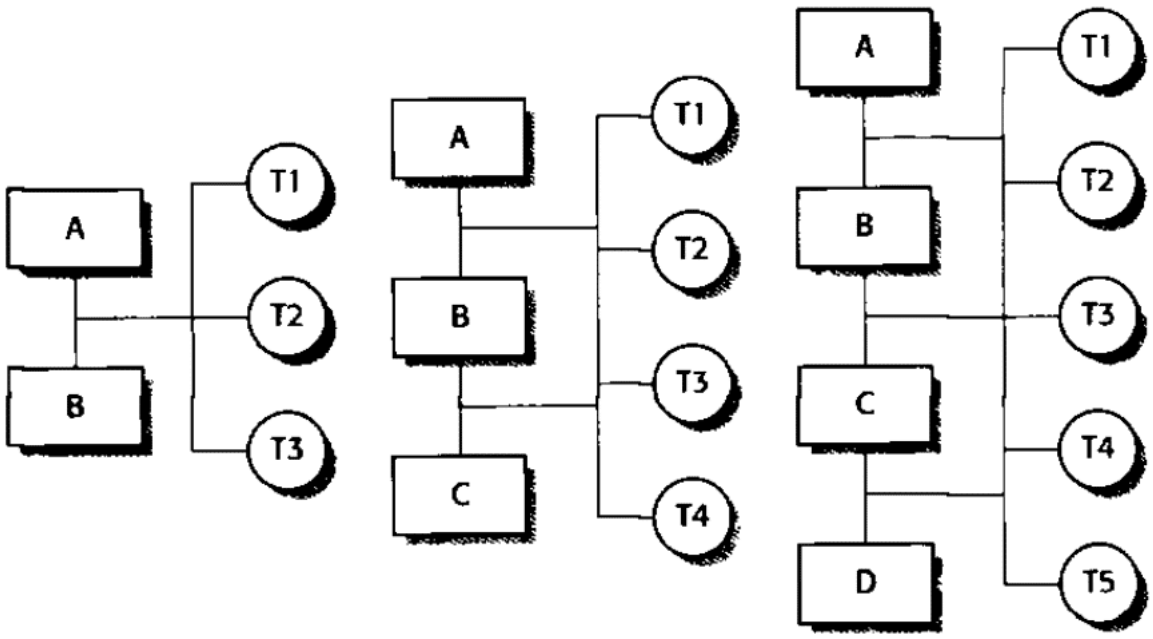
En la práctica, para muchos sistemas, la estrategia de integración es una mezcla de ambas, ya que se añaden en incrementos componentes de infraestructura y componentes funcionales. En ambas aproximaciones de integración, normalmente, tiene que desarrollarse un código adicional para simular otros componentes y permitir que el sistema se ejecute.

La principal dificultad que surge durante las pruebas de integración es la localización de los errores. Existen interacciones complejas entre los componentes del sistema y, cuando se descubre una salida anómala, puede resultar difícil identificar dónde ha ocurrido el error.

Para hacer más fácil la localización de errores, siempre debería utilizarse una aproximación incremental para la integración y pruebas del sistema. Inicialmente, debería integrarse una configuración del sistema mínima y probar este sistema.

Cuando se planifica la integración, tiene que decidirse el orden de integración de los componentes.

Figura 2: Integración de los componentes



Fuente: Sommerville, 2005, p. 496.

Pruebas de integración increméntales

A, B, C y D son componentes, y desde TI hasta T5 son conjuntos de pruebas relacionados de las características incorporadas al sistema. T I , T2 y T3 se ejecutan primero sobre un sistema formado por los componentes A y B (el sistema mínimo).

Si tales componentes revelan defectos, se corrigen. El componente C se integra y TI, T2 y T3 se repiten para asegurar que no ha habido interacciones no esperadas con A y B. Si surgen problemas en estas pruebas, esto significa, probablemente, que se deben a las interacciones con el nuevo componente. Se localiza el origen del problema y se simplifica, así, la localización y reparación de defectos. El conjunto de pruebas T4 se ejecuta también sobre el sistema. Finalmente, el componente D se integra y se prueba utilizando las pruebas existentes y nuevas (T5).

Pruebas de entregas

Las pruebas de entregas son el proceso de probar una entrega del sistema que será distribuida a los clientes. El principal objetivo de este proceso es incrementar la confianza del suministrador en que el sistema satisface sus requerimientos. Si es así, este puede entregarse como un producto o ser entregado al cliente. Para demostrar que el sistema satisface sus requerimientos, tiene que mostrar que posee la funcionalidad especificada, el rendimiento y la confiabilidad correctos y que no falla durante su uso normal.

Las pruebas de entregas son, normalmente, un proceso de pruebas de caja negra en las que las pruebas se derivan a partir de la especificación del sistema. El sistema se trata como una caja negra cuyo comportamiento solo puede ser determinado al estudiar sus entradas y sus salidas relacionadas.

Otro nombre para esto es **pruebas funcionales**, debido a que al probador solo le interesa la funcionalidad y no la implementación del *software*.

> **Entrada de datos → Sistema → Resultados de la prueba**

Pruebas de rendimiento

Una vez que un sistema se ha integrado completamente, es posible probar las propiedades emergentes del sistema, tales como rendimiento y fiabilidad. Las pruebas de rendimiento tienen que diseñarse para asegurar que el sistema pueda procesar su carga esperada. (Por ejemplo: debe procesar 1000 operaciones en dos horas).

Esto normalmente implica planificar una serie de pruebas en las que la carga se va incrementando regularmente hasta que el rendimiento del sistema se hace inaceptable.

Este tipo de pruebas tienen dos funciones:

1. Prueba el comportamiento del fallo de ejecución del sistema. Pueden aparecer circunstancias a través de una combinación no esperada de eventos en donde la carga sobre el sistema supere la máxima carga anticipada. En estas circunstancias, es importante que un fallo de ejecución del sistema no provoque la corrupción de los datos o pérdidas inesperadas de servicios de los usuarios. Las pruebas de estrés verifican que las sobrecargas en el sistema provocan «fallos ligeros» en lugar de colapsarlo bajo su carga.

2. Sobrecargan el sistema y pueden provocar que se manifiesten defectos que, normalmente, no serían descubiertos. Aunque se puede argumentar que es improbable que estos defectos causen fallos de funcionamiento en un uso normal, puede haber combinaciones inusuales de circunstancias normales que las pruebas de estrés pueden reproducir.

Pruebas de componentes

Las pruebas de componentes (a menudo denominadas pruebas de unidad) son el proceso de probar los componentes individuales en el sistema. Este es un proceso de pruebas de defectos, por lo que su objetivo es encontrar defectos en estos componentes. Tal y como se indicó en la introducción, para la mayoría de los sistemas, los desarrolladores de componentes son los responsables de las pruebas de componentes.

Existen diferentes tipos de componentes que pueden probarse en esta etapa:

1. Funciones individuales o métodos dentro de un objeto.

2. Clases de objetos que tienen varios atributos y métodos.

3. Componentes compuestos formados por diferentes objetos o funciones. Estos componentes compuestos tienen una interfaz definida que se utiliza para acceder a su funcionalidad.

Las funciones o métodos individuales son el tipo más simple de componente y sus pruebas son un conjunto de estas rutinas con diferentes parámetros de entrada. Pueden utilizarse las aproximaciones para diseñar los casos de prueba, descritos en la sección siguiente y para diseñar las pruebas de las funciones o métodos.

Cuando se están probando clases de objetos, deberían diseñar las pruebas para proporcionar cobertura para todas las características del objeto. Por lo tanto, las pruebas de clases de objetos deberían incluir lo siguiente:

1. Las pruebas aisladas de todas las operaciones asociadas con el objeto.

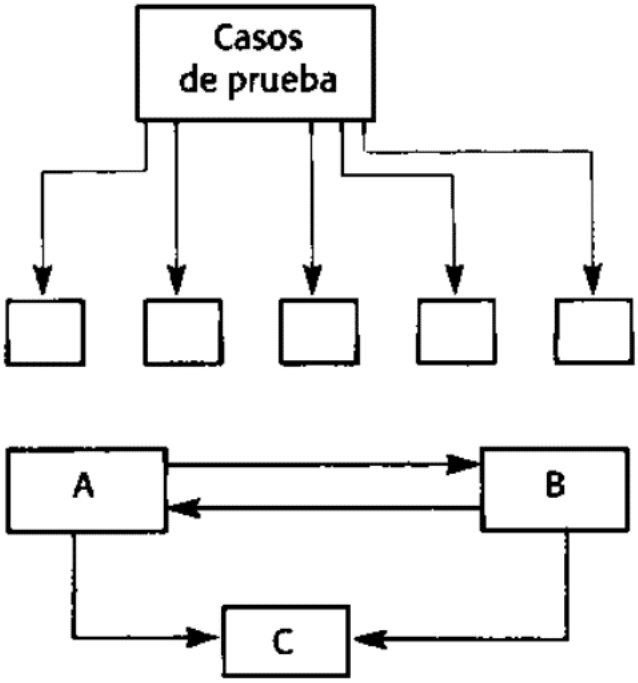
2. La asignación y consulta de todos los atributos asociados con el objeto.

3. La ejecución del objeto en todos sus posibles estados. Esto significa que deben simularse todos los eventos que provocan un cambio de estado en el objeto.

Pruebas de interfaces

Muchos componentes en un sistema no son simples funciones u objetos, sino que son componentes compuestos formados por varios objetos que interactúan.

Figura 3: Pruebas de interfaces con objetos que interactúan



Fuente: Sommerville, 2005, p. 502.

Supongamos que los componentes A, B y C se han integrado para formar un componente más grande o subsistema. Los casos de prueba no se aplican a componentes individuales, sino a la interfaz del componente compuesto que se ha creado combinando estos componentes.

Las pruebas de interfaces son particularmente importantes para el desarrollo orientado a objetos y basado en componentes. Los objetos y componentes se definen por sus interfaces y pueden ser reutilizados en combinación con otros componentes en sistemas diferentes.

Los errores de interfaz en el componente compuesto no pueden detectarse probando los objetos individuales o componentes. Los errores en el componente compuesto pueden surgir debido a interacciones entre sus partes.

Existen diferentes tipos de interfaces entre los componentes del programa y, consecuentemente, distintos **tipos de errores de interfaces** que pueden producirse:

1. **Interfaces de parámetros:** son interfaces en las que los datos, o algunas veces referencias a funciones, se pasan de un componente a otro en forma de parámetros.

2. **Interfaces de memoria compartida**: son interfaces en las que un bloque de memoria se comparte entre los componentes. Los datos se colocan en la memoria por un subsistema y son recuperados

desde aquí por otros subsistemas.

3. **Interfaces procedentes**: son interfaces en las que un componente encapsula un conjunto de procedimientos que pueden ser llamados por otros componentes. Los objetos y los componentes reutilizables tienen esta forma de interfaz.

4. **Interfaces de paso de mensajes:** son interfaces en las que un componente solicita un servicio de otro componente mediante el paso de un mensaje. Un mensaje de retorno incluye los resultados de la ejecución del servicio. Algunos sistemas orientados a objetos tienen esta forma de interfaz, así como los sistemas cliente-servidor.

Los errores de interfaces son una de las formas más comunes de error en sistemas complejos. Estos errores se clasifican en tres clases:

1. **Mal uso de la interfaz.** Un componente llama a otro componente y comete un error en la utilización de su interfaz. Este tipo de errores es particularmente común con interfaces de parámetros en donde los parámetros pueden ser de tipo erróneo, pueden pasarse en el orden equivocado o puede pasarse un número erróneo de parámetros.

2. **No comprensión de la interfaz.** El componente que realiza la llamada no comprende la especificación de la interfaz del componente al que llama y hace suposiciones sobre el comportamiento del componente invocado. El componente invocado no se comporta como era de esperar y esto provoca un comportamiento inesperado en el componente que hace la llamada. Por ejemplo, puede llamarse a una rutina de búsqueda binaria con un vector no ordenado para realizar la búsqueda. En este caso, la búsqueda podría fallar.

3. **Errores temporales**. Se producen en sistemas de tiempo real que utilizan una memoria compartida o una interfaz de paso de mensajes. El productor de los datos y el consumidor de dichos datos pueden operar a diferentes velocidades. A menos que se tenga un cuidado particular en el diseño de la interfaz, el consumidor puede acceder a información no actualizada debido a que el productor de la información no ha actualizado la información de la interfaz compartida.

Desarrollo dirigido de pruebas

- Primero, se identifica el incremento de funcionalidad requerido.
- Aquí, usualmente debe ser pequeño y aplicable en pocas líneas del código.
- Luego, se escribe una prueba para esta funcionalidad y se implementa como una prueba automatizada.
- En esta instancia, esto significa que la prueba puede ejecutarse y reportarse, sin importar si aprueba o falla.
- Luego, se corre la prueba, junto con todas las otras pruebas que se implementaron.

- Como próximo paso, se implementa la funcionalidad y se opera nuevamente la prueba.
- Finalmente, una vez puestas en funcionamiento con éxito todas las pruebas, se avanza a la implementación de la siguiente funcionalidad.

Pruebas de usuario

- definir criterios de aceptación;
- establecer un plan de prueba;
- ejecutar las pruebas de aceptación;
- ver resultados de las pruebas de aceptación;
- realizar reportes de la prueba de aceptación.

CONTINUAR

Testing

Dentro de los *testing*, tenemos diferentes tipos de fallas

Fallas en el desarrollo

- **Fallas de datos**. Variables usadas antes de la inicialización. Variables declaradas, pero nunca usadas.
- Variables asignadas dos veces, pero nunca usadas entre asignaciones. Posibles violaciones de límites de arreglo. Variables no declaradas.
- **Fallas de control**. Código inalcanzable. Ramas incondicionales en ciclos.
- **Fallas entrada/salida**. Variables de salida dobles sin intervención de asignación. Comprende utilizar variables que no cumplen ninguna función.
- **Fallas de interfaz:**
  - incompatibilidad del tipo de parámetro;
  - incompatibilidad del número de parámetro;
  - no se usan resultados de funciones;
  - funciones y procedimientos no llamados.
- **Fallas de gestión de almacenamiento:**
  - aritmética de almacenamiento;
  - fuga de memoria.

La validación de requerimientos es el proceso de verificar que los requerimientos definan lo que quiere el sistema.

- ◯ Falso.

- ◯ Verdadero.

SUBMIT

Los requerimientos en el documento no deben estar en conflicto en una comprobación de validez.

- ◯ Falso.
- ◯ Verdadero.

SUBMIT

Existen diferentes técnicas de validación de requerimientos. Identificar las que son correctas de la siguiente lista:

- ☐ Generación de casos de prueba.
- ☐ Creación de prototipos.
- ☐ Revisiones de requerimientos.
- ☐ Identificación de requerimientos.
- ☐ Validación de requerimientos.

SUBMIT

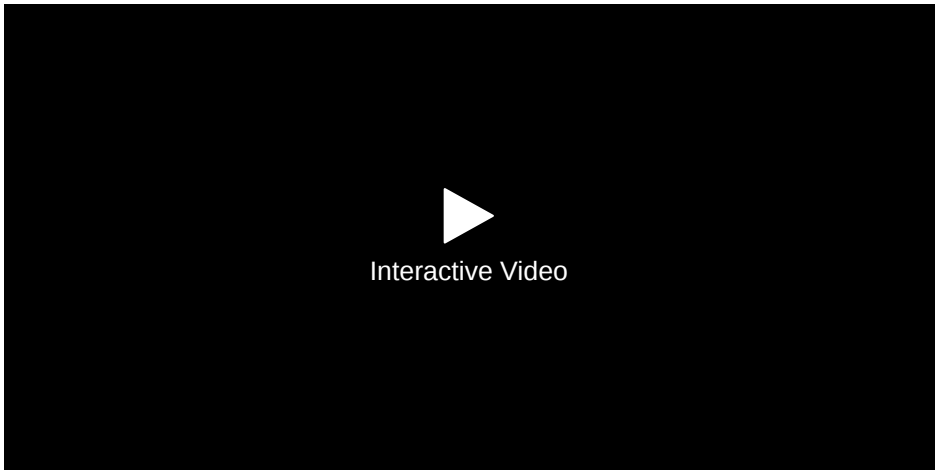
Variables asignadas dos veces, pero nunca usadas entre asignaciones. Esta es una falla: (identificar el tipo de falla en la siguiente lista)

- ◯ Fallas de datos.
- ◯ Fallas de control.
- ◯ Fallas de procesamiento.
- ◯ Fallas de salida.

SUBMIT

CONTINUAR

Video conceptual



CONTINUAR

Referencias

Sommerville, I. (2005). *Ingeniería de Software* (7.ma. ed.). Madrid, España: Editorial Pearson.

CONTINUAR